# SINIFLAR ARASI İLİŞKİLER

## Unified Modeling Language

Unified Modeling Language (UML) nesneye dayalı yazılım geliştirme ile birlikte gelişen bir modelleme aracıdır. Bu nedenle Nesne Yönelimli Programlama başlığında açıklanan nesne yönelimli birçok kavram, izleyen sayfalarda daha da pekişecektir. Adından da anlaşılacağı üzere nesneye dayalı problem çözmenin kalbi, model oluşturmaktır.

Model, gerçek dünyadaki karmaşık problemin esaslarını soyut olarak önümüze koyar. Buradan hareketle UML'i, basit bir dil, gösterim ya da sözdizimi (syntax) gibi algılamalıyız. UML'in, yazılımı nasıl geliştireceğimizi kesin olarak belirtmediği göz önüne alınmalıdır.

UML, bir grafik modelleme dili olup yazılım sistemlerini oluşturan ana elemanları (Ki bunlar İngilizce "artifact" olarak adlandırılıp ve el yapımı anlamına gelmektedir.) çeşitli şekillerden oluşacak şekilde tanımlamak için oluşturulmuş bir kurallar bütünüdür. Örnek verecek olursak, bir mimari projeye ilişkin maket ne ise yazılım için UML de aynı şeyi ifade eder. Yani UML, söz konusu yazılımın neyi yapacağını anlatan çeşitli şekillerin anlamlı bir şekilde bir araya getirilmesini sağlayan bir standarttır.

Yazılım mühendisliğiyle (software engineering) birlikte 1989 ve 1994 yılları arasında elliden fazla modelleme dili kullanılmaya başlanmıştır. Bu nedenle bu dönem yöntem savaşlarının yaşandığı bir dönem olarak adlandırılır. Bu yöntemlerin çoğu kendi sözdizimini oluşturmuş olmakla birlikte kullandıkları dil elemanları açısından birbirleriyle benzerlik göstermektedirler.

Doksanlı yılların ortalarında üç yöntem, daha güçlü ve yaygın hale gelmiştir. Bu yöntemlerin her biri, diğerlerinin içerdiği elemanları da içeriyordu ancak her birinin diğerine karşı çeşitli güçlü özellikleri bulunuyordu. Bu yöntemler:

- *Booch* yönteminin tasarım (design) ve kodlama (implementation) yönleri oldukça iyi idi. *Grady Booch* Ada diliyle oldukça çalışmıştı ve nesne yönelimli tekniklerin Ada diline uygulanması konusunda yaptığı çalışmalarla bu konuda önemli bir oyuncu olmuştur. Booch metodu güçlü olmasına rağmen model, birçok bulut şeklinden oluşmasından dolayı gösterim şekli oldukça sevimsizdi.
- Object Modeling Technique (OMT) yöntemi, *Jim Rumbaugh* tarafından geliştirilmiş olup, analiz ve veri merkezli bilgi sistemlerinin modellenmesinde oldukça iyi bir yöntemdir.
- Object Oriented Software Engineering (OOSE) modeli use-case olarak bilinen bir model olup *Ivar Jacobson* tarafından geliştirilmiştir. Use-case modeli, yazılımı yapılacak sistemin davranışlarını anlamaya yönelik gelişmiş güçlü bir tekniktir.

1994 yılında *Jim Rumbaugh* ve General Electric firmasından ayrılan *Grady Booch* birlikte Rational şirketini kurarak bir araya geldiler. Bu birlikteliğin amacı, kendi yöntemlerini bir araya getirecek yeni ve tek bir yöntem olan "Unified Method"u oluşturmak idi.

1995 yılında *Ivar Jacobson* da Rational şirketine katıldı. *Ivar Jacobson*'ın use-case'ler konusundaki fikirleriyle birlikte Rational şirketi, bugün Unified Modeling Language-UML olarak adlandırılan, yeni bir "Unified Method" geliştirdi. Üç kişiden oluşan bu grup "Three Amigos" olarak bilinir.

Yöntem savaşlarının ardından güçlü yazılım üreticileri bir araya gelerek OMG adında bir şirketler birliği kurdular[18]. Bu şirketler birliği 1997 yılında UML'e sahip çıkarak herkes tarafından kullanılan bir dil olmasını ve bir standart haline gelmesini sağlamıştır.

UML sadece, gerçek dünyadaki süreçlerin modelini ortaya koyan bir gösterim (notation) sistemidir. Süreçlerin standartlaştırılmasına yönelik bir dil değildir. UML ile ortaya çıkan modeller, süreçleri soyut (abstract) olarak tanımlayacaktır, ancak süreçlerin doğruluğu hakkında bir bilgi vermeyecektir. Yani, A şirketi için çalışan bir süreç, B şirketine uygulandığında felaketle sonuçlanabilir.

[18] OMG (Object Management Group), www.omg.org

Model, yukarıda da anlatıldığı üzere çözülecek probleme ilişkin bir soyutlamadır (abstraction). Etki alanı (domain) ise problemin yaşandığı gerçek dünyayı ifade eder.

Model, birbirlerine ileti (message) gönderen nesnelerden (object) oluşur. Bu modelde nesneler canlı (alive) olarak düşünülmelidir. Nesneler bir şeyler bilir (know) ve bildikleriyle bir şeyler yaparlar (do).

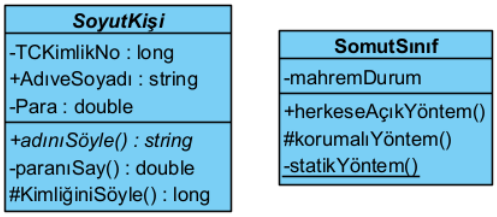
Modelde nesneler, nesne tanımında verilen özellik (property) ve alanları (field) gösteren niteliklere (attribute) sahiptirler. Nesnelerin gösterdikleri davranış (behavior) ya da geliştirdiği yöntemler (method) ise işlem (operation) olarak adlandırılır. Nesnenin özelliklerine ait o anki değerler ise nesnenin durumunu (state) belirler.

UML, sekiz tür diyagrama sahiptir ve bu kitapta sınıf diyagramlarına yer verilecektir[19].

# Sınıf Diyagramları

Sınıf diyagramları (class diagram), yazılımı geliştirilecek olan sisteme ait sınıfları (class) ve bu sınıflar arasındaki ilişkiyi (relationship) gösterir. Sınıf diyagramları durağandır (static). Yani sadece sınıfların birbiriyle etkileşimini (what interacts) gösterir, bu etkileşimler sonucunda ne olduğunu (what happens) göstermez.

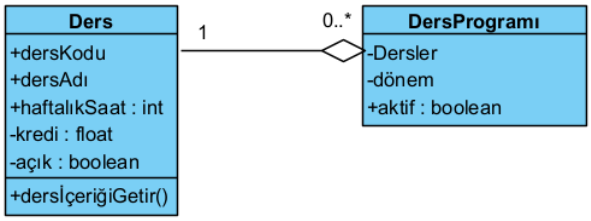
Sınıf diyagramlarında sınıflar, bir dikdörtgen ile temsil edilir. Bu dikdörtgen üç parçaya ayrılır. En üstteki birincisine sınıf ismi yazılır. Ortadaki ikincisinde nitelikler (attribute), en alttaki ve sonuncusunda ise ve işlemler (operation) yer alır.




***Şekil 24.*** *Örnek Bir Sınıf Diyagramı*

Sınıf isminin italik yazılması, sınıfın soyut (abstract class) sınıf olduğunu gösterir. Benzer şekilde işlemlerin yer aldığı kısımda, yöntemlerin italik yazılması halinde, ilgili yöntemin soyut yöntem (abstract method) olduğunu gösterir.

Sınıflarda erişim belirleyiciler (access modifier), özel karakterlerle birbirinden ayrılır. Burada "-" karakteri mahrem (private), "+" karakteri umuma açık (public), "#" karakteri ise korumalı (protected) olduğunu anlatmaktadır. Statik üyeler (static member), altı çizili olarak gösterilir.



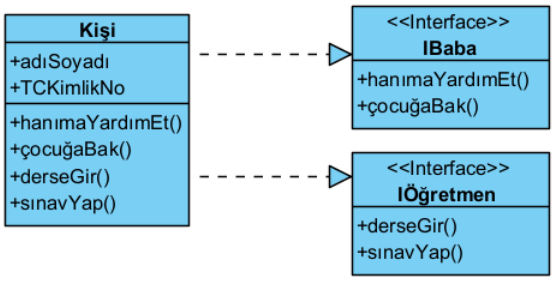

***Şekil 25.*** *Sınıf Diyagramlarında Sınırlama ve Çokluk Örneği*

Çokluk (multiplicity), ilişkide bir sınıfın örnek (instance) sayısını gösterir. İlişkiyi gösteren çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk bir sonraki başlıkta verilen, iş birliği (association), bileşim (composition) ve bütünleşmeye (aggregation) uygulanır. Çokluk örnekleri:

- `0..1` Sıfır ya da 1 örnek.
- `0..` Veya `*` Örnek sayısı sınırsızdır.
- `1` Yalnızca 1 örnek

[19] www.togethersoft.com/services/practical_guides/umlonlinecourse/index.html

– `1..` En az bir örnek




***Şekil 26.*** *Sınıf Diyagramlarında Ara yüz Gerçekleştirme*

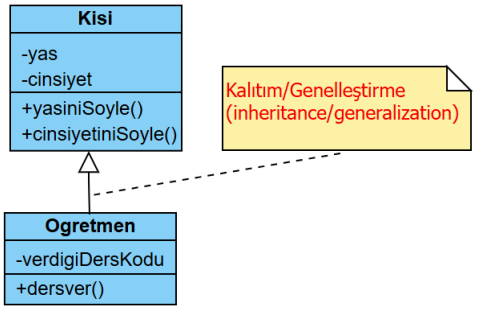
Ara yüz gerçekleştirme (interface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar lolipop diyagramı olarak da adlandırılır.

# Sınıflar Arası İlişkiler

Nesneleri imal ettiğimiz sınıflar arasında ya ilişki yoktur ya da aşağıda sıralanan altı çeşit ilişki vardır;

## Genelleştirme İlişkisi

Genelleştirme (generalization), taban sınıfla (base class) ile türemiş sınıf (derived class) arasındaki kalıtım (inheritance) ilişkisidir. Örneği *Kalıtım* başlığında verilmiştir. Aşağıda Kişi sınıfı ile bu sınıftan miras alan Öğretmen sınıfının UML diyagramı verilmiştir. Ara yüzler (interface) veya roller de benzer şekilde gösterilir.




***Şekil 27.*** *Genelleştirme İlişkisi UML Diyagramı*

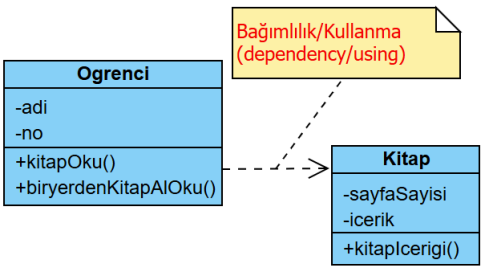
## Bağımlılık İlişkisi

Bağımlılık (dependency) ya da bir başka deyişle kullanma (using), bir sınıf ile diğer arasındaki geçici ilişkidir. Burada kullanan sınıf istemci (client) diğer sınıf ise sağlayıcı (supplier) olarak adlandırılır. İki türlü bağımlılık vardır; Birincisinde istemci sınıf, sağlayıcıyı sınıfı parametre olarak kullanır (dependency as a parameter). İkincisinde ise istemci sınıf, sağlayıcı sınıftan örnekleme yapar (dependency as an instantiation).

```
#include <iostream>
using namespace std;
class Kitap {
public:
    string kitapIcerigi() {
        return icerik;
    }
    Kitap(int pSayfaSayisi, string pIcerik): sayfaSayisi(pSayfaSayisi), icerik(pIcerik) {
    }
private:
    string icerik;
    int sayfaSayisi;
```

```
};
class Ogrenci {
public:
    void kitapOku() {
        Kitap* kitap=new Kitap(150,"C++ ile Nesne Yönelimli Programlama Notları");
        /* pKitap nesnesine örnekleme olarak bağımlılık:
           dependency as an instantiation */
        cout << "kitap imal edilip kullanılıyor..." << endl;
        string notlar=kitap->kitapIcerigi();
        //...
        delete kitap;
    }
    void biryerdenKitapAlOku(Kitap* pKitap) {
        /* pKitap nesnesine parametre olarak bağımlılık:
           dependency as a parameter */
        cout << "kitap dışardan alınıp kullanılıyor..." << endl;
        string okunacak=pKitap->kitapIcerigi();
        //...
    };

    Ogrenci(string pAdi,int pNo): adi(pAdi),no(pNo) {
    }
private:
    string adi;
    int no;
};
int main() {
    Ogrenci* ogrenci= new Ogrenci("Ali",12000);
    ogrenci->kitapOku();
    Kitap* kitap=new Kitap(350,"Kurumsal yazılım Geliştirme");
    ogrenci->biryerdenKitapAlOku(kitap);
    delete kitap;
    delete ogrenci;
}
/* Program Çıktısı:
kitap imal edilip kullanılıyor...
kitap dışardan alınıp kullanılıyor...

...Program finished with exit code 0
*/
```

Örnekteki Öğrenci ve Kitap sınıfı arasındaki bağımlılık ilişkisi aşağıdaki UML diyagramında verilmiştir.




***Şekil 28.*** *Bağımlılık İlişkisi UML Diyagramı*

## İş Birliği ya da Ortaklık İlişkisi

İş Birliği (association) ya da ortaklık, iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkisidir. Bu ilişkide, bir sınıf diğer sınıftan nesne örnekler (imal eder), kullanır ve öldürür. İlişkinin sürekli olabilmesi için kullanılan sınıftan bir değişken alan (field) olarak tanımlanır. İki türü vardır;

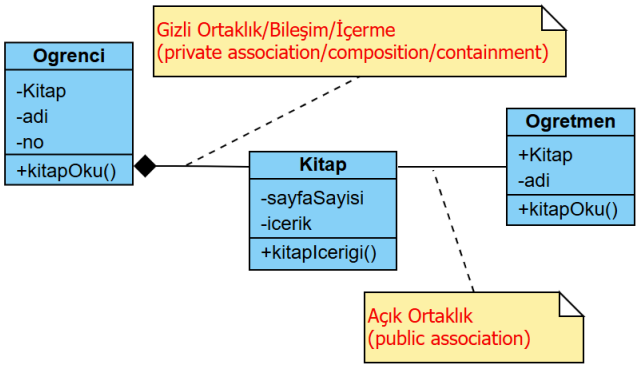
– Açık ortaklık (public association): Kullanılan sınıfa ilişkin değişken `public` erişimi ile tanımlanır.

– Gizli ortaklık (private association), içerme (containment) ya da bileşim (composition): Kullanılan sınıfa ilişkin değişken `private` erişimi ile tanımlanır ve dışarıdan erişime izin verilmez.

```
#include <iostream>
using namespace std;
class Kitap {
public:
    string kitapIcerigi() {
        return icerik;
    }
    Kitap(int pSayfaSayisi, string pIcerik): sayfaSayisi(pSayfaSayisi), icerik(pIcerik) {
    }
private:
    string icerik;
    int sayfaSayisi;
};
class Ogrenci { // kitapsız öğrenci olmaz! (private association to kitap)
private:
    string adi;
    int no;
    Kitap* ptrKitap;
public:
    Ogrenci(string pAdi,int pNo): adi(pAdi),no(pNo) {
        ptrKitap=new Kitap(150,"C ile Yapısal Programlama");
        //kitap nesnesi ile ogrenci nesnesi birlikte imal ediliyor:
    }
    void kitapOku() {
        cout << "Öğrenci ile birlikte imal edilen kitap kullanılıyor..." << endl;
        cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << endl;
        //...
    }
};
class Ogretmen { // Kitapsız öğretmen olmaz! Ama kitabını değiştirebilir.
//public association to kitap
private:
    string adi;
    Kitap* ptrKitap;
public:
    Ogretmen(string pAdi): adi(pAdi) {
        //kitap nesnesinin referansı ile ogretmen nesnesi birlikte imal ediliyor:
        ptrKitap=new Kitap(200,"C++ ile Nesne Yönelimli Programlama");
    }
    // Kitap* üyesi get ve set yöntemleriye public hale getiriliyor.
    void setKitap(Kitap* pKitap) {
        ptrKitap=pKitap;
    }
    Kitap* getKitap() {
       return ptrKitap;
    }
    void kitapOku() {
        cout << "Öğretmen ile birlikte imal edilen kitap kullanılıyor..." << endl;
        cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << endl;
        //...
    }
};
int main() {
    Ogrenci* ogrenci=new Ogrenci("Ali",12000);
    ogrenci->kitapOku();

    Ogretmen* ogretmen=new Ogretmen("Ilhan");
    ogretmen->kitapOku();
```

```
    Kitap* ogretmeninOkuduguKitap=ogretmen->getKitap();
    cout << "Orgetmenin Okuduğu Kitaba Main İçinden Ulaşılabiliyor;" << endl
         << "Okunan Kitap:" << ogretmeninOkuduguKitap->kitapIcerigi() << endl;

    Kitap* baskaKitap=new Kitap(250,"Kurumsal Yazılım Geliştirme");
    ogretmen->setKitap(baskaKitap);
    ogretmen->kitapOku();
    delete ogrenci, ogretmen, baskaKitap;
}
/*Program Çıktısı:
Öğrenci ile birlikte imal edilen kitap kullanılıyor...
Okunan Kitap:C ile Yapısal Programlama
Öğretmen ile birlikte imal edilen kitap kullanılıyor...
Okunan Kitap:C++ ile Nesne Yönelimli Programlama
Orgetmenin Okuduğu Kitaba Main İçinden Ulaşılabiliyor;
Okunan Kitap:C++ ile Nesne Yönelimli Programlama
Öğretmen ile birlikte imal edilen kitap kullanılıyor...
Okunan Kitap:Kurumsal Yazılım Geliştirme

...Program finished with exit code 0
*/
```

Yukarıdaki örnekte Öğrenci ile Kitap arasındaki gizli ortaklık ile Öğretmen ve Kitap arasındaki açık iş birliği aşağıdaki UML diyagramında gösterilmiştir;




***Şekil 29.*** *Açık ve Gizli İş Birliği UML Diyagramı*

UML diyagramlarında, çift yönlü ortaklıklar iki oka sahip olabilir veya hiç ok olmayabilir ve tek yönlü ortaklıkta veya kendi kendine ortaklık bir oka sahiptir.

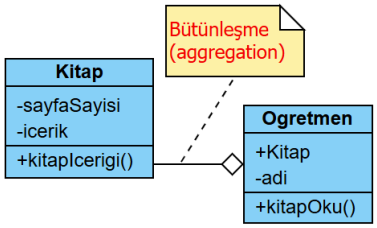
## Bütünleşme İlişkisi

Bütünleşme (aggregation), ilişkisi de iki farklı sınıf arasındaki sürekli olan bütün-parça (whole-part) ilişkisidir. Açık ortaklığa (public association) benzer, farklı olarak bu ilişkide kullanılan sınıfa ait başka yerde imal edilmiş nesneyi ya da hazır olan nesneyi kullanır. Tek fark kullanılacak sınıf nesnesinin kullananın iradesi dışında yaratılmış olmasıdır.

```
#include <iostream>
using namespace std;
class Kitap {
public:
    string kitapIcerigi() {
        return icerik;
    }
    Kitap(int pSayfaSayisi, string pIcerik): sayfaSayisi(pSayfaSayisi), icerik(pIcerik) {
    }
private:
    string icerik;
    int sayfaSayisi;
};
```

```
class Ogretmen {
private:
    string adi;
    Kitap* ptrKitap;
public:
    Ogretmen(string pAdi,Kitap* ppKitap): adi(pAdi),ptrKitap(ppKitap) {
        //öğretme nesnesi kitap nesnesi olamadan imal edilemez.
        //Bir nesne imal edilmeden bi kitap nesnesi olmalı
        //aggregation to kitap
    }
    //Kitap* durumu get ve set metodlarıyla public hale getiriliyor.
    void setKitap(Kitap* pKitap) {
        ptrKitap=pKitap;
    }
    Kitap* getKitap() {
        return ptrKitap;
    }
    void kitapOku() {
        cout << "Öğretmen ile birlikte imal edilen kitap kullanılıyor..." << endl;
        cout << "Okunan Kitap:" << ptrKitap->kitapIcerigi() << endl;
        //...
    }
};
int main() {
    Kitap kitap= Kitap(150,"C++ ile Nesne Yönelimli Programlama");
    Ogretmen ilhan("Ilhan",&kitap);
    ilhan.kitapOku();
}
/* Program Çıktısı:
Öğretmen ile birlikte imal edilen kitap kullanılıyor...
Okunan Kitap:C++ ile Nesne Yönelimli Programlama

...Program finished with exit code 0
*/
```

Yukarıdaki örnekte verilen Öğretmen ve Kitap arasındaki bütünleşme ilişkisi aşağıdaki UML diyagramında gösterilmiştir.

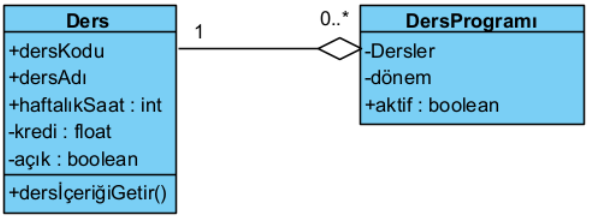



***Şekil 30.*** *Bütünleşme UML Diyagramı*

# Sınıflar Arası İlişkilerde Diyagramlar

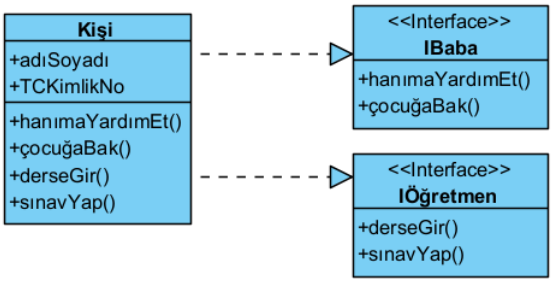
Sınıflar arası ilişkilerde birleşme çizgilerinde rakamlar da bulunabilir; Çokluk (multiplicity), ilişkide bir sınıfın örnek (instance) sayısını gösterir. İlişkiyi gösteren çizginin sonunda bir numara olarak gösterilir. Bu numara, mümkün olabilecek örnek sayısıdır. Çokluk, açık ortaklık (public association), gizli ortaklık (private association) ve bütünleşmeye (aggregation) uygulanır;

- `0..1` Sıfır ya da 1 örnek.
- `0..` veya `*` Örnek sayısı sınırsızdır.
- `1` Yalnızca 1 örnek
- `1..` En az bir örnek




***Şekil 31.*** *Sınıf Diyagramlarında Çokluk Örneği*

Ara yüz gerçekleştirme (interface realization) işlemi, ucunda üçgen olan kesikli çizgi ile gösterilir. Üçgenin sivri köşesi gerçekleştirilen ara yüzü gösterir. Çemberler, ara yüzleri (interface) göstermenin bir başka yoludur. Bu durumda daha kısa ve etkili anlatıma sahip diyagramlara sahip oluruz. Bu tip basitleştirilmiş diyagramlar lolipop diyagramı olarak da adlandırılır.




***Şekil 32.*** *Sınıf Diyagramlarında Ara yüz gerçekleştirme*

# Sınıf Tasarlama İlkeleri (SOLID)

Nesne yönelimli programlama (object oriented programming); Kodumuzda değişim yönetimi (change management), gösterici kullanımı yerine referansları kullanarak güvenli kod (safe code) aynı kodları tekrar yazmak (duplicate code) yerine kodların yeniden kullanımı (reusing) sağlayan aşağıdaki özellikleri kullanırız;

- Kod küçüldüğü için kalıtım (inheritance),
- Veri ve kontrol soyutlaması (abstraction) yaparak daha güvenli bileşen tasarladığımız için sarmalama (encapsulation),
- Yazılım mimarisinin daha esnek ve genişleyebilir olması için çok biçimlilik (polymorphism).

Ayrıca yazılım geliştirmede;

- Zayıf bağlaşımın (loosely coupling) sağlanması için yazılımın bir noktasında olabilecek değişikliğin, diğer kısımlarda değişiklik gerektirmeyecek şekilde tasarlanması gerekir. Yani yapılan değişiklik, çorap söküğü gibi yazılımın diğer kısımlarını değiştirmemelidir.
- Aynı çağrışım kümesine ilişkin bütün bileşenler bir arada geliştirilmeli diğer kümelerle karıştırılmamalıdır (seperation of concern). Yani bileşenler arasında yüksek uyum (high cohesion) olmalıdır.

İşte yeniden kullanım, zayıf bağlaşım ve yüksek uyum sağlamak için programcı, nesne yönelimli programlama özellikleriyle birlikte temel ilkelere (principle) uyar. Programcı kodu tasarlarken, yazarken ve yeniden düzenlerken bu ilkeleri aklında tutarsa; Kod çok daha temiz, genişletilebilir ve test edilebilir olur. İlkelerin İngilizce baş harflerinin alınarak isimlendirilir.

## Tek Sorumluluk İlkesi

Bu ilke ile (**S**ingle Responsibility Principle-SRP) sınıflar, tasarlanırken birden fazla işten sorumlu tutulmamalıdır. Mümkün olduğunca bir işle ilgili olarak tasarlanmalıdır. Bir sınıfın tek bir şey yapması gerektiğini ve dolayısıyla değişmesi için tek bir nedeninin olması gerektiğini belirtir.

```
// Kötü Örnek: Birden fazla sorumluluk
class Employee {
    void calculatePay() { /* ... */ }
```

```
    void saveToDatabase() { /* ... */ }
    void generateReport() { /* ... */ }
};

// İyi Örnek: Her sınıf tek sorumluluk
class Employee {
    // Sadece employee verisi
};
class PayrollCalculator {
    void calculate(const Employee& emp) { /* ... */ }
};
class EmployeeRepository {
    void save(const Employee& emp) { /* ... */ }
};
class ReportGenerator {
    void generate(const Employee& emp) { /* ... */ }
};
```

## Açık-Kapalı İlkesi

Bu ilke ile (**O**pen Closed Principle-OCP) sınıflar, değişikliklere kapalı (closed for modification) eklentilere açık (open for extension) şekilde tasarlanmalıdır. Bu şekilde değişikliklere açık genişleyebilir modüllere sahip oluruz. Değişikliklere kapalı olmak, mevcut bir sınıfın kodunu değiştirmemek anlamına gelirken, eklentilere açık olma yeni işlevler eklemek anlamına gelir.

```
// Eklentiler için açık, değişiklik için kapalı
class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};
class Circle : public Shape {
private:
    double radius_;
public:
    double area() const override {
        return M_PI * radius_ * radius_;
    }
};
class AreaCalculator {
public:
    //std::vector ve std::unique_ptr için şablonlar başlığını inceleyiniz
    double totalArea(const std::vector<std::unique_ptr<Shape>>& shapes) {
        double total = 0;
        for (const auto& shape : shapes) {
            total += shape->area();
        }
        return total;
    }
};
```

## Liskov İkame İlkesi

Bu ilkede (**L**iskov Substitution Principle-LSP) alt sınıflar türediği sınıfların yerine konulabilmelidir. Bu prensipten ilk olarak Barbar Liskov tarafından bahsedilmiştir. Alt sınıflar, türediği sınıf davranışlarıyla kullanılabilmelidir. Burada amaç, alt sınıfların daha çok değişebileceği göz önüne alınarak değişimlerden an az şekilde etkilenmektir.

```
// Alt sınıflar üst sınıf yerine kullanılabilmeli
class Bird {
public:
    virtual ~Bird() = default;
```

```
    virtual void eat() { /* ... */ }
};
class FlyingBird : public Bird {
public:
    virtual void fly() { /* ... */ }
};
class Sparrow : public FlyingBird {
    void fly() override { /* Can fly */ }
};
class Penguin : public Bird {
    // Uçamaz, FlyingBird'den türemez
};
```

Türemiş sınıfından bir nesneyi, Taban sınıfından bir nesne bekleyen herhangi bir yönteme parametre olarak geçirdiğimizde yöntemin herhangi bir tuhaf çıktı vermemesi gerektiği anlamına gelir. Bu beklenen davranıştır, çünkü kalıtım kullandığımızda alt sınıfın, üst sınıfın sahip olduğu her şeyi miras aldığını varsayarız. Alt sınıf davranışı genişletir ancak asla daraltmaz. Dolayısıyla bir sınıf bu ilkeye uymadığında, tespit edilmesi zor bazı kötü hatalara yol açar.

## Ara Yüzleri Ayırma İlkesi

Bu ilke ile (**I**nterface Segregation Principle-ISP) genel amaçlı ara yüzler yerine istemciye bağlı ara yüzler tasarlanmalıdır. Yani roller, birbirinden ayrı olacak şekilde tanımlanmalıdır. Birden fazla rol tek bir rol altında olacak şekilde düşünülmemelidir. Bu nedenle buna rollerin ayrılması prensibi (Role Segregation Principle-RSP) olarak da adlandırılır.

```
// Küçük, işe özel (spesific) interface'ler
class IPrintable {
public:
    virtual void print() = 0;
};
class IScannable {
public:
    virtual void scan() = 0;
};
class IFaxable {
public:
    virtual void fax() = 0;
};
class SimplePrinter : public IPrintable {
    void print() override { /* ... */ }
};
class MultiFunctionPrinter : public IPrintable,
                             public IScannable,
                             public IFaxable {
    void print() override { /* ... */ }
    void scan() override { /* ... */ }
    void fax() override { /* ... */ }
};
```

## Bağımlılıkları Ters Çevirme İlkesi

Bu ilke (**D**ependency Inversion Principle-DMP), sınıflarımızın somut sınıflara ve fonksiyonlara değil, ara yüzlere veya soyut sınıflara bağlı olması gerektiğini belirtir. Nesne yönelimli programlamanın en önemli ilkesidir. Aslında bu ilke ile açık-kapalı ilkesi doğrudan birbiriyle ilişkilidir.

```
// Soyutlamalara bağımlı ol, somut sınıflara değil
class IDatabase {
public:
    virtual ~IDatabase() = default;
    virtual void save(const std::string& data) = 0;
};
```

```
class MySQLDatabase : public IDatabase {
    void save(const std::string& data) override {
        // MySQL implementation
    }
};
class PostgreSQLDatabase : public IDatabase {
    void save(const std::string& data) override {
        // PostgreSQL implementation
    }
};
class UserService {
private:
    // std::unique_ptr için şablonlar başlığını inceleyiniz
    std::unique_ptr<IDatabase> database_;
public:
    UserService(std::unique_ptr<IDatabase> db)
        : database_(std::move(db)) {}

    void saveUser(const std::string& user) {
        database_->save(user);
    }
};
```